

Lab1:Datalab from 陈南晖 25301020039

执行 ./check_ops.py bits.c 之后的截图:

```
root@ics:~/cnh_lab1# ./check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 14/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 12/56 operators
rotateRightBits: 11/16 operators
roundEvenPow2: 21/24 operators
midpointTowardFirst: 19/32 operators
isBetweenEitherOrder: 46/48 operators
mul5Sat: 29/30 operators
classifyAdd3: 42/52 operators
floatScaleThreeHalves: 38/60 operators
floatRoundEven: 25/65 operators
float_i2f: 26/40 operators
bitCount: 24/40 operators
bitReverse: 33/34 operators
All 19 functions passed operator checks.
```

执行 ./btest 之后的截图:

```
root@ics:~/cnh_lab1# ./btest
Score  Rating  Errors  Function
1      1        0    signMask
2      2        0    bitXor
3      3        0    negativePart
4      4        0    copyByteWithin
4      4        0    logicalShift
4      4        0    swapNibblePairs
4      4        0    secondLowestZeroBit
5      5        0    oddParity
5      5        0    rotateRightBits
5      5        0    roundEvenPow2
5      5        0    midpointTowardFirst
7      7        0    isBetweenEitherOrder
7      7        0    mul5Sat
7      7        0    classifyAdd3
7      7        0    floatScaleThreeHalves
10     10        0    floatRoundEven
10     10        0    float_i2f
10     10        0    bitCount
10     10        0    bitReverse
Total points: 110/110
root@ics:~/cnh_lab1#
```

实验思路

P1 signMask

将数字 1 左移 31 位，使唯一的 1 移动到最高符号位，其余位补 0，得到 0x80000000 掩码。操作数 1/2。

P2 bitXor

基于德摩根定律，仅用 ~ 和 & 实现异或：排除两 bit 同时为 1 和同时为 0 的情况，剩余一 0 一 1 的场景即为异或结果。操作数 7/8。

P3 negativePart

算术右移 31 位得到符号掩码，负数为全 1、非负为全 0。与补码表示的 $-x$ 相与，负数返回相反数，非负数返回 0。操作数 5/6。

P4 copyByteWithin

字节偏移量 = 编号左移 3（等价乘 8），提取源字节，构造目标字节清零掩码，擦除原值后移入新字节并合并。操作数 9/12。

P5 logicalShift

先执行默认算术右移，再构造高位为 0、低位为 1 的掩码，清除算术右移补入的符号位，实现高位强制补 0 的逻辑右移。操作数 7/20。

P6 swapNibblePairs

用 0x0F 移位拼接出每字节低 4 位掩码，取反得高 4 位掩码。分别移位后按位或，实现每个字节内部高低半字节交换。操作数 11/18。

P7 secondLowestZeroBit

按位取反将找 0 转为找 1，清除最低位的 1 后，提取剩余最低位的 1，对应原数倒数第二个 0 的位置。用 $\sim + 1$ 替代减号。操作数 5/8。

P8 oddParity

通过 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 逐次折叠异或，将 32 位中 1 的个数奇偶性压缩到最低位，取反后得到偶校验结果。操作数 10/56。

P9 rotateRightBits

拆分两部分：右移 n 位的低位段，和左移到高位的溢出段。右移段用掩码清除符号扩展，两段按位或完成循环右移。操作数 10/16。

P10 roundEvenPow2

提取余数和向下取整基数，余数大于中点则进位；等于中点时判断商的奇偶性，奇数则进位保证最终商为偶数。用补码替代减号。操作数 20/24。

P11 midpointTowardFirst

用 $(x \& y) + ((x \hat{y}) \gg 1)$ 公式无溢出求平均。出现半整数时，结合 x 的符号与大小判断是否加 1，使结果向 x 靠拢。操作数 18/32。

P12 isBetweenEitherOrder

异号看符号、同号用减法安全比较大小，覆盖 $a < b$ 和 $b < a$ 两种区间场景，补充端点相等的情况，合并后返回 0/1 结果。操作数 43/48。

P13 mul5Sat

拆成 $(x \ll 2) + x$ 。先判断左移是否丢信息，再判断加法是否溢出。若溢出，按 x 符号饱和到 INT_MAX 或 INT_MIN ；否则返回计算结果。操作数 29/30。

P14 classifyAdd3

用两个 32 位变量模拟 64 位加法。分别算 $x+y$ 和 $(x+y)+z$ 的低 32 位与进位，得到高 32 位。根据高 32 位和低 32 位符号判断真实和是否超出正/负范围，返回 1、-1 或 0。操作数 42/52。

P15 floatScaleThreeHalves

拆符号、阶码、尾数，NaN/Inf 原样返回。规格化数取 24 位有效数乘 3，除以 2 时舍入到偶，超 24 位则右移并加阶码。非规格化数 $\text{frac} \times 3$ 后舍入，不足 2^{23} 保持非规格化，否则升为阶码 1 的规格化数。操作数 34/60。

P16 floatRoundEven

拆符号、阶码、尾数，NaN/Inf 原样返回。按阶码 E 分类：E<-1 舍入到 0；E=-1 根据尾数决定舍入到 0 或 1；E>=23 已是整数； $0 \leq E < 23$ 时提取整数和小数部分，按最近偶数舍入，再规格化为浮点。操作数 25/65。

P17 float_i2f

0 返回 0。取符号和绝对值，找最高位 bit，指数 $E = \text{bit} + 127$ 。若 $\text{bit} > 23$ ，右移 $\text{bit} - 23$ 位得 24 位有效数，按最近偶数舍入，进位则调整指数；否则左移补齐。去掉隐藏位，组装符号、指数、尾数。操作数 26/40。

P18 bitCount

整数规则禁用大常量，先用 0x55、0x33、0x0F、0xFF 移位拼出掩码。然后并行加法树：每 2 位、4 位、8 位、16 位、32 位分组相加，最终得到 1 的总数。操作数 24/40。

P19 bitReverse

采用分治逐层交换：依次完成相邻 1bit 互换、2bit 块互换、4bit 块互换、8bit 字节互换、16 位半字互换，最终实现 32 位整体反转。用 0x55、0x33、0x0F、0xFF 小常量移位拼接各级掩码，符合 d1c 常量限制。操作数 27/34。

感谢 DeepSeek 对我的很多提问都进行了耐心的解答!!!

对本实验的感受：

好难。。。许多题目很难想出来（TAT）。